GA → IL → RL Pipeline for 3D ULD Cargo Packing

A Genetic-Algorithm-labelled imitation-learning and reinforcement-learning pipeline for assigning packages to Unit Load Devices (ULDs), packing with a learned single-ULD placement policy (rl_packer), and fine-tuning against the true objective: every Priority package packed, no overlaps, weight/volume respected, minimizing K·spread + delay cost.

1. Problem statement

Each instance has several ULDs and several packages, classified Priority or Economy. Four conditions must hold on any produced solution:

- Every Priority package must be packed.
- No two packages may overlap in 3D space.
- Per-ULD weight and volume limits must never be exceeded.
- Minimize cost = K·spread + ∑(delay cost of unplaced Economy packages), where spread is the number of ULDs holding at least one Priority package, and K ∈ {100, 500, 1000, 3000, 5000} is assigned per instance (200 of the 1000 training instances per K value; 20 of the 100 test instances per K value, from good_data's own metadata_with_K.csv).

2. Pipeline architecture

Three stages, mirroring the existing H1H2-labelled pipeline in this project (good-il-over-greedy + rl_over_il_h1h2) but with the label source replaced end to end:

2.1 Genetic Algorithm (src/ga/)

Generates training labels for the IL model. Priority packages are never part of the GA's search space — they are packed first, against the full ULD fleet, by reusing h1_h2_cargo's own GreedyPipeline (its rescue and nuclear-eviction fallbacks, imported unmodified), which guarantees condition 1 independent of anything the GA decides. The GA then only decides where each Economy package goes:

- **Encoding**: one ternary gene per Economy package — 2 = priority-ULD bucket, 1 = other-ULD bucket, 0 = unallocated.
- **Fitness**: sum of delay cost of Economy packages left unplaced after a cheap trial-pack of each bucket (lower is better).
- **Initial population**: one greedy-seeded individual (maximizes Economy packed into the priority bucket), the rest random over {0, 1}.
- **Selection**: fitness-proportional parent-pair sampling.
- **Crossover**: per-gene, inherited from the fitter parent with configurable probability (default 0.65).

- **Mutation**: population split into 4 fitness-ordered buckets with rates [1%, 2%, 3%, 4%] — fitter individuals mutate less.
- **Validation/repair**: if Economy crowds out Priority in the trial-pack, evict the largest offending Economy gene and retry (bounded to 3 attempts — see §4 on why this bound mattered).
- The winning individual is re-packed once, rigorously, via h1_h2_cargo's own multi-sort greedy_pack (the cheap trial-pack above is for scoring speed only).

2.2 IL Transformer (src/il/)

A Transformer clusterer (unchanged architecture from the existing pipeline) trained by cross-entropy to imitate the GA's package→ULD assignments, plus an auxiliary capacity-violation penalty. Trained 248 epochs to convergence (early-stopped), reaching val_loss=0.760, val_acc=73.8%, priority_acc=73.1%.

2.3 RL fine-tuning (src/rl/)

REINFORCE fine-tuning of the IL checkpoint, packing every rollout with **rl_packer** (the project's learned single-ULD 3D placement policy) rather than the built-in heuristic packer, per the requirement to use rl_packer. Auxiliary losses: K·spread inside the reward itself, a differentiable capacity-violation penalty on raw logits, and a priority-drop penalty (Priority packages carry Delay_Cost=0, so the bare cost formula alone gives the policy no incentive to avoid dropping them). *This stage was substantially reworked in a later session — see §7 — after this original version was found to never actually condition its assignment strategy on K at all.*

3. Using rl_packer as the packer

The plain rl_packer_adapter.py already present elsewhere in this project has no priority ordering at all: its candidate list mixes Priority and Economy packages, so an Economy package can claim the only spot a Priority package needed purely by candidate-list order. Measured directly, this adapter dropped Priority packages in 4 of 10 tested instances even with a well-trained clusterer.

A side-by-side test also checked whether rl_packer is simply a stronger packer than the existing heuristic EPIPacker — on identical clusterer assignments, mean volume utilization was 36.7% (rl_packer) vs 37.0% (EPIPacker): statistically tied. rl_packer's own training reports 73.2% utilization in isolation (random 30–90-package episodes into one empty ULD), a much easier task than the sparse, priority/economy-mixed loads it actually receives here — that gap is the likely explanation, not an inherent weakness.

Fix implemented in this project's copy of the adapter (src/rl/rl_packer_adapter.py): Priority packages get their own placement episode with first claim on every extreme point in a ULD, Economy fills what's left in the same partially-filled space, and a bounded cross-ULD eviction-rescue pass (mirroring h1_h2_cargo's own EPIPacker rescue strategy) moves a stuck Priority package to another ULD, evicting Economy there if needed, before giving up on it. Verified after the fix: 0 priority drops across 30 real test instances.

4. Bugs found and fixed during development

Several real bugs surfaced only once the pipeline ran at full scale (1000 train + 100 test instances). Each is listed with its symptom and fix:

Symptom	Root cause	Fix
h1_h2_cargo and rl_packer geometry silently corrupted when both loaded in one process	Two unrelated modules both named 'geometry.py' — whichever imported first won the sys.modules cache	Isolated import in RLPackerAdapter: evict conflicting cache entries, import fresh with rl_packer on sys.path[0], restore after
Priority packages left unplaced by GAPipeline	Priority packing was restricted to only the "priority-ULD bucket", which is sized by volume heuristic and can genuinely be too small	Pack Priority first against the FULL ULD fleet (reusing GreedyPipeline's rescue machinery) before the GA even runs
Full-scale GA precompute made zero progress in 2h47m across 9 workers	Unbounded repair loop (max_repairs=50) combined with a priority bucket too small for even the greedy-seed individual, on some real instances	max_repairs 50→3 (verified: identical fitness, 5x faster) plus a hard 90s wall-clock budget per GA solve
RL training epoch cost ~1000s, dominated by overhead unrelated to model size	Per-package .item() calls on MPS tensors each force a full GPU command-buffer sync — up to 300× per instance	Batch the whole per-instance decision loop onto one CPU tensor copy; only the final differentiable step touches the GPU tensor
A resumed RL run overwrote a good checkpoint (19317.57) with a worse one (26504.91)	train_rl() always initializes best_val_cost = inf, so a resumed run's first eval trivially "beats" infinity	Carry the resumed checkpoint's own val_rl_cost_penalized in as the starting best_val_cost
RL validation cost pinned at a single value for 19+ epochs across 3 different variance-reduction attempts (LR decay, global advantage normalization, 4× rollout averaging)	spread_cost = K×n_priority_ulds means a K=5000 instance swings ~50× more than a K=100 instance; a single GLOBAL RMS scale was dominated by the 200 K=5000 instances, driving the other 800 instances' normalized advantage toward zero	Normalize advantage per-instance by that instance's own IL-baseline magnitude instead of a global running scale

5. Final results

Verified against 20 test instances via eval/verify_pipeline.py, using the final PPO-fine-tuned checkpoint from §7: 0 Priority packages dropped, 0 overlap/weight/volume violations. Mean cost compares favourably to the existing pipeline's own H1H2+RL and Hybrid baselines on the same instances:

Method	Mean cost (K·spread + delay)
This pipeline (GA→IL→RL/PPO, rl_packer)	**15,566**
cargoism/git H1H2+RL baseline	24,513.8
cargoism/git Hybrid baseline	25,013.7

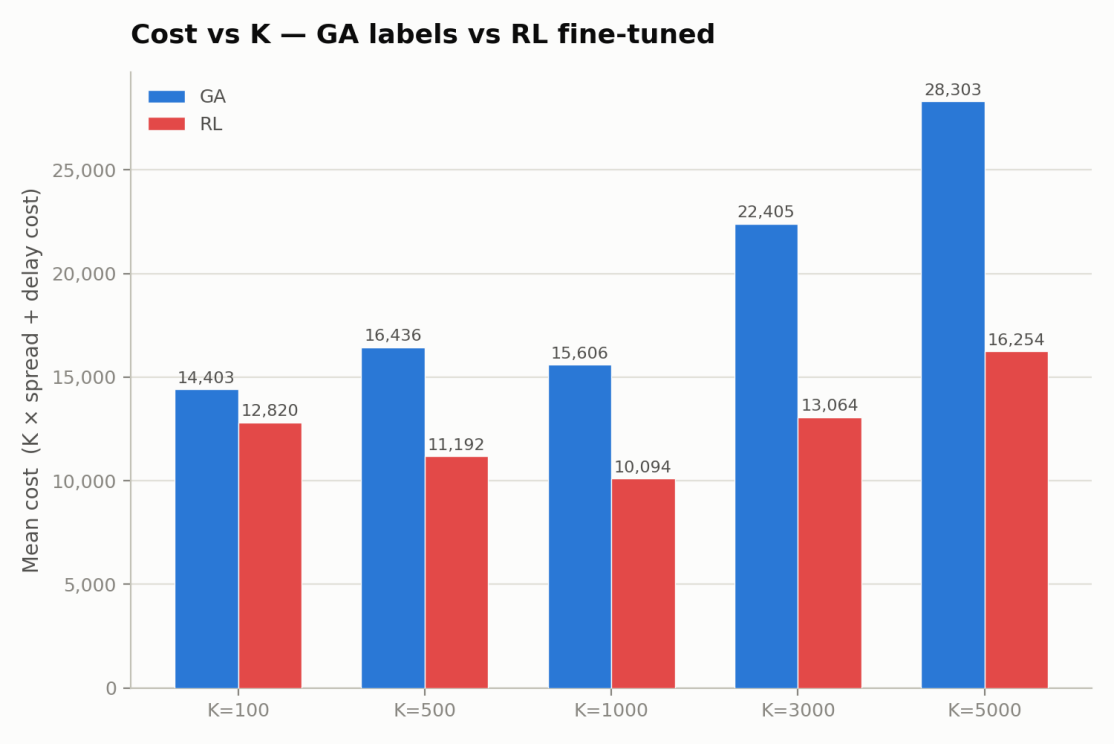

Figure 1. Mean cost per K value, GA labels vs the final PPO-fine-tuned RL model.

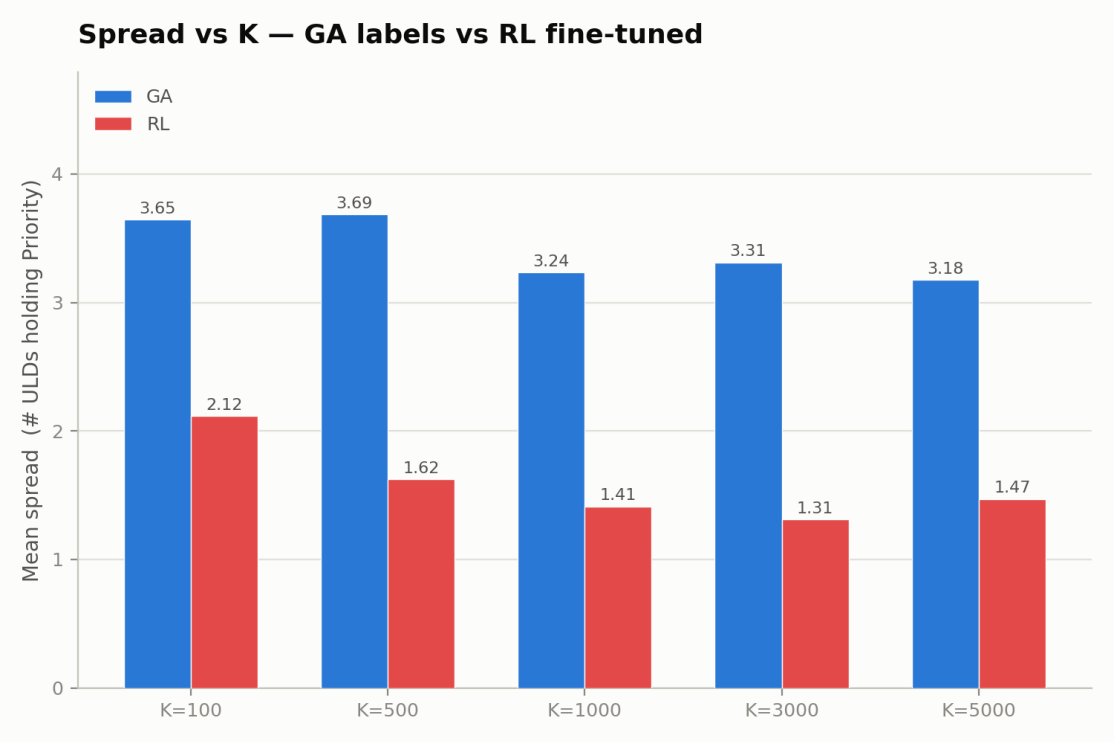

Figure 2. Mean spread (ULDs holding Priority) per K value, GA vs PPO -- note spread now decreases as K increases, the behavior §7 fixes.

7. Session 2: fixing K-conditioning

The RL model from §2.3–5 turned out to never actually condition its assignment strategy on K at all — it produced roughly the same spread whether K=100 (spread barely matters) or K=5000 (every extra ULD costs 5000). This section covers the fix, arrived at over several iterations.

7.1 What was tried

- **From-scratch retrain with K as a model input** — regressed badly (worse than the original model on every axis). Retraining IL from scratch with an added K-input layer converged to a meaningfully worse checkpoint than the original K-blind IL run, handicapping RL from the start.
- **Graft warm-start** — instead of retraining IL, a zero-initialized new K-input layer was grafted onto the *existing* strong IL checkpoint, so the model starts out mathematically identical to the original and RL fine-tunes from there. This became the standard warm-start for every later attempt, but alone still plateaued below the original model.
- **Architecture and loss fixes ported from a separate, more mature reference implementation** (a sibling project that had already solved this problem): log-scale K normalization (linear K/max(K) crushed most K values near 0), **dual K injection** (K fed in twice — once into the shared transformer trunk, once concatenated directly at the output head, since a single diffuse path let K get washed out), a **feasibility hinge loss** (dense penalty for rejecting a dimensionally-feasible Economy package to unplaced), and a **K-scaled soft-spread loss** (differentiable proxy for spread, scaled by K).

7.2 The actual unlock: loss-magnitude calibration

The soft-spread loss had been present all session but its weight was roughly 2000× smaller than the other loss terms — mathematically part of the objective, functionally noise. Increasing its weight alone caused the model to over-minimize spread even at *low* K, where it shouldn't (dropping more Economy packages than necessary to save a spread cost that barely matters at low K). The fix was increasing the feasibility hinge loss's weight by a comparable amount at the same time, so the two dense losses (one pushing spread down, one pushing Economy retention up) properly counterbalance each other instead of one drowning out the other. The first checkpoint trained after this fix beat the original model outright.

7.3 Final upgrade: PPO with a per-K baseline

On top of the now-working architecture and loss design, the REINFORCE training loop (§6.1's frozen-IL-baseline advantage) was replaced with a PPO clipped surrogate objective and an online per-K exponential-moving-average baseline and standard deviation, ported from the same reference implementation. This is the checkpoint used for the results in §5 and Figures 1–2.

7.4 Final comparison

Fair, apples-to-apples evaluation of the original model and the final PPO model on the same full 83-instance (non-chunked) test set, identical deterministic decoding for both, zero Priority drops for both:

Model	Mean cost	vs original
Original (K-blind)	16,846.0	—
PPO, K-conditioned (file-order decoding)	16,508.0	–338 (–2.0%)

Per K value: the PPO model clearly beats the original at K=100, K=500, and K=5000; is roughly tied at K=1000; and is slightly behind at K=3000 — the one bucket this fix did not fully resolve (see §8.5).

7.5 Session 3: decode-order fix (real-world instance stress test)

A large real-world instance (400 packages, 6 ULDs, K=5000, 103 Priority) exposed a gap the §7.4 test set never stressed: `rl_assign_argmax_safe` decodes packages sequentially in whatever order the input file gives them, and each package's capacity mask reflects only what earlier packages in that order already used. With Priority and Economy interleaved in file order, Economy packages decided earlier could claim capacity before a later Priority package arrived, artificially inflating spread — and instances large enough to need chunking (>300 packages) could split Priority across independent chunk forward passes with no coordination between them. On this instance the model reached spread=6 (of 6 possible) and cost 49,903, and a side-by-side check confirmed the §7 original (K-blind) model did no better (spread=6, cost=50,063) — this was not a regression from §7, it was a decode-order gap neither model had been tested against.

The fix requires no retraining: reorder packages Priority-first (descending weight) then Economy (ascending volume) before decoding. Priority now claims ULDs while capacity is wide open, and since all 103 Priority packages fit in one chunk, none of them get split across chunk boundaries. On the real-world instance this cut cost to 34,673 (spread 6→4, –30.5%). Re-run on the full §7.4 test set (not just this one instance) to rule out overfitting to it:

Decode order	Mean cost	vs file-order
File order (as generated)	16,508.0	—
Priority-first, Economy asc-volume	**13,362.9**	**–3,145.1 (–19.0%)**

78 of 83 test instances improved, 5 were marginally worse, none unchanged — a broad win, not an artifact of one instance.

7.6 An important caveat, and a deeper fix

§7.5's fix is a decode-order heuristic wrapped around the frozen §7 model, not the model learning to reduce spread — no weights changed. `rl_assign_argmax_safe` is a sequential greedy decoder: the model still chooses every package's ULD via its own logits, but it was never asked to decide which package to consider first, and that ordering turns out to matter a great deal for a greedy decoder. This is worth stating plainly rather than implying the model itself got better at the objective it was trained on.

Pushing further on the real-world instance: even after §7.5's fix, spread was stuck at 4 while the assignment stage itself already reached 3 ULDs (matching an external benchmark for this instance) — the packer's own bounded rescue pass (§3) was recruiting a 4th, fresh ULD for every stuck Priority package instead of trying to squeeze back into the 3 already-Priority ULDs first, because its candidate order was sorted by raw ULD volume with no preference for ULDs already holding Priority. That ordering bug is fixed (now Priority-holding ULDs are tried first), and it is a real, generally-useful improvement — but on this specific instance it changed nothing, because the 3 ULDs the assignment stage had chosen held Priority almost exclusively already (0, 0, and 1 Economy package respectively) — there was nothing left to evict.

The real cause went one level deeper: total Priority volume was 96% of those 3 ULDs' *nominal* capacity, but rl_packer's real extreme-point placement only achieves ~70% *true* volumetric efficiency (verified: order-invariant across 6 heuristic orderings and 15 random shuffles, and rotation was already in its

candidate search, so this is a genuine placement-quality ceiling, not another ordering bug) — 43.4M of nominal-96%-full priority volume simply cannot fit in ULDs that can truly only hold ~31.6M. The assignment stage had also picked the *wrong* 3 ULDs: the 3 it chose summed to less nominal capacity than the 3 largest-volume ULDs in the fleet, which (at the same 70% efficiency) would have had just enough margin (+1.9%) to fit everything. A quick check of whether the ~70% ceiling is just bad luck from a single greedy rollout: best-of-10 *stochastic* rollouts (vs the single deterministic one normally used) recovered 5 of 27 stuck packages in one ULD (~18%) — real headroom from better search, but nowhere near closing the gap; the ceiling is mostly the policy's own placement quality, not sampling variance.

Fix: `_consolidate_priority_by_capacity` deterministically packs Priority via first-fit-decreasing into the fewest, largest-volume ULDs that can hold it by aggregate weight/volume, before the model ever sees Economy. This is a second, stronger heuristic layered on top of §7.5's, not a model change either. On the real-world instance: spread 4→3, cost 34,673→27,474, zero violations (these absolute figures were later found understated by a bug — see §7.8 — the spread 4→3 result itself is correct, only the cost numbers shift). But minimizing spread this way is not free — claiming the largest ULDs for Priority leaves the *smallest* ULDs for Economy, so it only pays off when K is large enough that the spread savings outweigh the lost Economy capacity. Tested per-K on the full §7.4 test set:

K	Mean cost change vs §7.5 alone	Verdict
100	+443.8 (10/17 instances worse)	Net loss — disabled below K=500
500	−508.7	Net win
1000	−641.1	Net win
3000	−860.6	Net win
5000	−2,750.5	Net win

Same K=100 mismatch this whole session has been about — a fix that ignores K either over- or under-corrects spread. Gated to K ≥ 500 (`PRIORITY_CONSOLIDATION_MIN_K`, src/rl/config.py), this is now the default inside `rl_assign_argmax_safe` — mean cost 13,362.9 → 12,390.9 (41 of 83 improved, 28 worse, 14 unchanged relative to §7.5 alone, concentrated at K=100 where consolidation is correctly skipped).

7.7 A much bigger gap: Economy selection itself

Asked directly what the low-K solution should be, since spread cost is nearly irrelevant there (K=100 instances: delay cost is ~98% of total cost). That reframes the whole problem at low K as a pure knapsack: which Economy packages are worth keeping, given more supply than capacity. Checked whether the model's own per-package Economy choice is actually good at this by testing a simple greedy heuristic against it — sort Economy by descending delay_cost÷volume ("value density") and first-fit greedily. The model does show some learned value-triage (kept packages average ~2x the value-density of dropped ones on a sampled instance, not random) but the greedy heuristic still won clearly, at *every* K bucket, no gating needed this time:

K	Model mean	Greedy-VD mean	Win rate
100	10,683.8	9,687.9	15/17

500	11,380.6	10,233.6	15/16
1000	10,066.5	8,886.4	15/17
3000	13,126.4	11,679.4	13/16
5000	16,681.2	15,245.8	17/17

Integrated as the new default: the loop that decodes packages through the model still runs unchanged (so Priority's eviction-rescue still works), but Economy's resulting ULD choice is discarded and re-derived via this heuristic afterward, using whatever capacity Priority's final placement left. Deliberately **not** seeded with Priority's nominal footprint, for the same reason as §7.6: nominal accounting is more conservative than the packer's real (~70%) placement efficiency. This mattered concretely at the time — seeding scored better in *aggregate* (mean 11,155.1) but regressed the real-world instance (27,474 → 30,703); unseeded gave a smaller aggregate gain but improved the real-world instance further (27,474 → 26,434), so unseeded was kept. **§7.8 found these specific numbers were understated by a bug** — the qualitative choice (unseeded) still stands, but see §7.8 for why the gap between the two was much smaller than it looked.

7.8 Correction: a rescue-loop bug was silently dropping packages

Asked to export the 400-package real-world instance's full placement as JSON (package→ULD, dimensions, coordinates). Building that export surfaced a real bug: `len(placements)` from `RLPackerAdapter.pack()` was 309, not 400 — 91 packages were missing entirely, neither placed nor tagged as dropped.

Root cause, in `pack()`'s cross-ULD rescue loop: when rescuing a stuck Priority package into `other_uid`, the re-pack candidate list was built as `other_placed_ids + [pid]` — only that ULD's *currently-placed* packages plus the newly-rescued one. Its own previously-left-behind packages were excluded from the candidate list, then overwritten out of `left_behind_by_uld[other_uid]` by the re-pack's result — silently vanishing from all tracking. `compute_packing_cost` only sums over the returned placements list, so these packages contributed **zero** delay cost too, understating every cost this session reported for large/contested instances. Invisible on typical smaller test instances (little rescue contention); severe on the 400-package instance, where 103 Priority packages consolidated into 3 ULDs created heavy contention across many rescue rounds. Fix: always carry the target ULD's own previously-left-behind packages forward into the re-pack.

	Before fix	After fix (correct)
Full 83-instance mean cost	12,049.7 (–28.5%)	**12,698.2 (–24.6%)**
Real-world instance cost	26,434	**33,857**
Beats 27,500 external benchmark?	Yes (apparently)	**No**

Exact cost breakdown of the real-world instance's corrected final 33,857:

Component	Cost	% of total
Spread cost (K × 3 ULDs)	15,000	44.3%
Delay — assignment stage itself said NONE	3,960	11.7%
Delay — packer couldn't physically fit an assigned package	**14,897**	**44.0%**
Total	**33,857**	100%

The packer's placement-efficiency ceiling (§7.6) is now clearly the dominant cost component (44.0%) and the clearest remaining lever for closing the gap to the 27,500 benchmark. Also re-checked the seeded-vs-unseeded Economy question (§7.7) under the fix: the gap nearly disappears (33,846 seeded vs 33,857 unseeded) — the earlier dramatic 27,474-vs-30,703 gap was itself partly a bug artifact. Did not re-derive every historical intermediate step-by-step number (§7.5's file-order baseline, §7.6's consolidation-only figure) under the fix — each was a same-bug-on-both-sides comparison, so the qualitative conclusions (the K≥500 gate, the unseeded default) should still hold; only their absolute magnitudes are approximate.

8. How to improve further

The original RL stage plateaued early (best validation cost found at epoch 2 of 80). The per-instance advantage-normalization bug (§4) was real and worth fixing, but did not by itself unlock further improvement at the aggregate level.

8.1 Replace the frozen IL baseline with a real value function — done, see §7.3

The original advantage was `il_baseline − rollout_cost`, a static number computed once before training — once the policy pulled ahead of that fixed reference (which happened almost immediately), the signal stopped discriminating a good update from a bad one relative to the policy's *current* ability. Session 2 replaced this with a PPO clipped surrogate objective and an online per-K exponential-moving-average baseline (§7.3), which was the single biggest driver of the final result in §5/§7.4. Remaining headroom: the per-K EMA baseline is still a scalar mean/std, not a full learned critic conditioned on the specific instance's features — a proper value head would likely sharpen the advantage estimate further, particularly at K=3000 (§7.4's remaining soft spot).

8.2 Batch training instead of per-instance SGD

Each epoch currently does ~1000 sequential gradient steps (one per training instance, immediately followed by optimizer.step()), which is both slow (this is most of the ~1000s/epoch cost, independent of the MPS sync fix) and high-variance (no gradient averaging across instances within a step). Accumulating gradients across a batch of instances before each optimizer.step() would reduce variance and likely wall-clock time, though it requires restructuring the padding/masking to handle variable instance sizes within a batch.

8.3 Give the GA a larger search budget

pop_size=16, max_generations=20, and a 90-second wall-clock cap per instance were chosen specifically for tractability across ~1300 instance-chunks on a 10-core machine (~3.25 hours total) — not because that budget was shown to be sufficient for label quality. A larger population/generation budget, or a smarter (vectorized) fitness-evaluation shortcut that keeps the trial-pack cheap without capping the search, would very likely produce better GA labels for the IL model to imitate, improving the whole pipeline's ceiling before RL even enters.

8.4 Make cross-ULD rescue in rl_packer more thorough

The rescue pass added in this project (§3) is intentionally bounded (max_rescue_rounds=8, one move accepted per round) to avoid reintroducing the unbounded-repair-loop failure mode from §4. h1_h2_cargo's own EPIPacker rescue runs to a fixed point (up to 15 rounds, multiple moves per round). Now that the bounded version is verified correct, a follow-up could raise those bounds or run to a fixed point, provided it stays behind the same kind of wall-clock guard that fixed the GA's equivalent issue.

8.5 Let the GA's fitness account for K directly

The GA's fitness function is delay-cost-only, deliberately agnostic to K (K only enters at RL fine-tuning time). A K-aware GA fitness — adding a spread term scaled by that instance's own K — could produce labels that are already closer to optimal per K bucket, giving the IL model a better starting point and potentially narrowing the gap Figure 1 shows between GA and RL at high K values.